```
from google.colab import files
uploaded = files.upload()
```

Choose files Screenshot … 143119.png

Screenshot 2026-07-13 143119.png(image/png) - 177156 bytes, last modified: 13/07/2026 - 100% done

```
Saving Screenshot 2026-07-13 143119.png to Screenshot 2026-07-13 143119.png
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Use the uploaded image path
image_path = 'Screenshot 2026-07-13 143119.png'

img = cv2.imread(image_path)

if img is None:
    print(f"Error: Could not load image from {image_path}. Please make sure the file exists and is ac
else:
    rows, cols = img.shape[:2]

    # Define the affine transformation matrix for a translation
    # The values 1000 and 500 were causing the image to be translated completely out of view
    # when combined with original image dimensions (rows, cols) being the warpAffine size.
    # Let's use smaller, more visible translation values for demonstration.
    # For a translation of (tx, ty), the matrix is: [[1, 0, tx], [0, 1, ty]]
    tx = 50  # Translate 50 pixels right
    ty = 30  # Translate 30 pixels down
    M = np.float32([[1, 0, tx], [0, 1, ty]])

    # Perform the affine transformation
    # The output image size (cols, rows) ensures the transformed image fits within the original dimen
    affine_img = cv2.warpAffine(img, M, (cols, rows))

    # Convert images from BGR to RGB for correct display with matplotlib
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
    affine_img_rgb = cv2.cvtColor(affine_img, cv2.COLOR_BGR2RGB)

    # Display the original and transformed images using matplotlib
    plt.figure(figsize=(12, 6))

    plt.subplot(1, 2, 1)
    plt.imshow(img_rgb)
    plt.title('Original Image')
    plt.axis('off')

    plt.subplot(1, 2, 2)
    plt.imshow(affine_img_rgb)
    plt.title(f'Affine Transformed (Tx={tx}, Ty={ty})')
    plt.axis('off')

    plt.show()

    # If you still want to save the transformed image to disk, uncomment the line below:
    # cv2.imwrite('Affine_Transformed.jpg', affine_img)
```